

CHAPTER 6.3: HOMOMORPHIC ENCRYPTION

FOUNDATIONS OF DISTRIBUTED SYSTEMS

Isabel Wagner

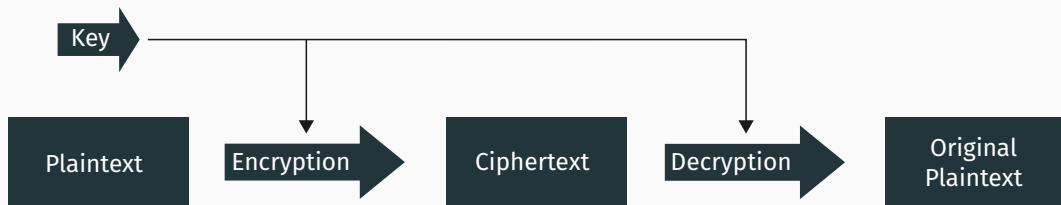
Fall semester 2025

University of Basel

1. Cryptography refresher
2. Homomorphism
3. Partially homomorphic encryption (PHE)
4. Somewhat homomorphic encryption (SHE)
5. Fully homomorphic encryption (FHE)

1. CRYPTOGRAPHY REFRESHER

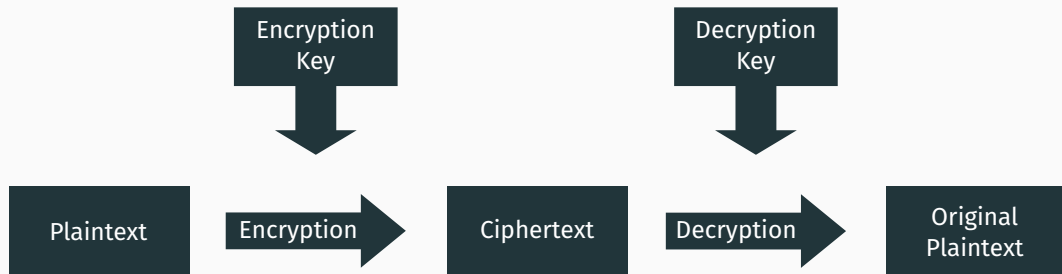
SYMMETRIC CRYPTOGRAPHY¹



- How often do you use symmetric cryptography? Which algorithms do you know?
- To define a symmetric cryptosystem: need to define functions for key generation, encryption, and decryption

¹C. Pfleeger, S. L. Pfleeger, and J. Margulies, *Security in Computing*, 5th edition. Pearson, 2015.

ASYMMETRIC (PUBLIC-KEY) CRYPTOGRAPHY²



- How often do you use public-key cryptography? Which algorithms do you know?
- To define a public-key cryptosystem: need to define functions for key generation, encryption, and decryption

²C. Pfleeger, S. L. Pfleeger, and J. Margulies, *Security in Computing*, 5th edition. Pearson, 2015.

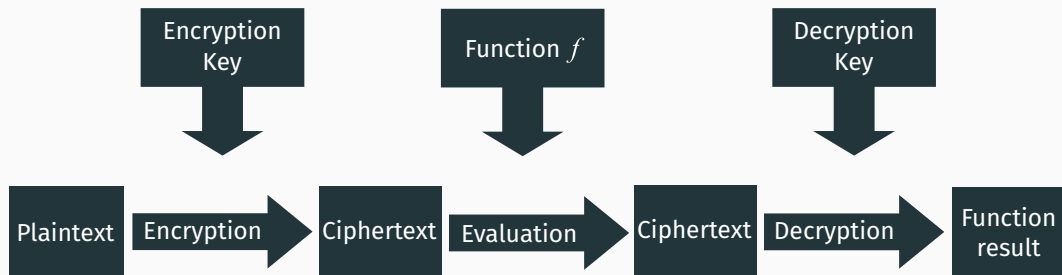
- Ensure confidentiality (and integrity, authentication via hashes, MACs, signatures)
- for data in transit
 - Communication between Alice and Bob
 - Adversary controls the communication medium
- for data at rest
 - Storage of data by Alice
 - Adversary controls the storage medium
- But not: for **data during computation**

SETTING & GOALS FOR HOMOMORPHIC ENCRYPTION³

- We have data that we want to keep private
- We want to process the data (e.g., with some method from data science)
- But we do not have enough processing power!
- If we have a **fully homomorphic encryption** scheme, we can:
 - Encrypt the data
 - Give the encrypted data and our analysis code to a third party
 - Third party can run the computations without the secret decryption key
 - Third party returns the result to us
 - We can decrypt the result
- We can outsource computations while keeping the data private!

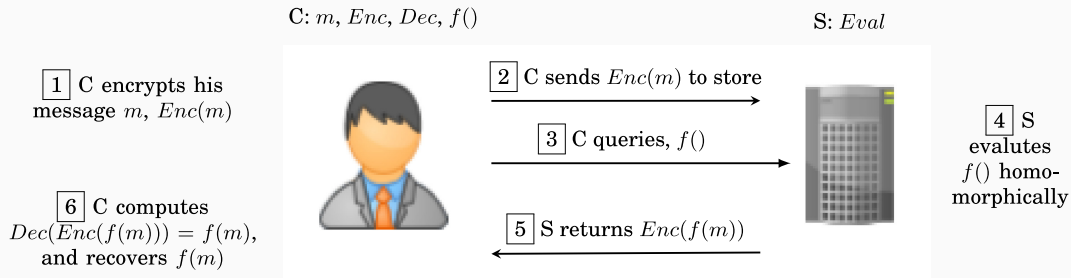
³C. Gentry, "Computing arbitrary functions of encrypted data," *Communications of the ACM*, vol. 53, no. 3, pp. 97–105, Mar. 2010. doi: 10.1145/1666420.1666444. [Online]. Available: <https://dl.acm.org/doi/10.1145/1666420.1666444>.

HOMOMORPHIC ENCRYPTION



- To define a homomorphic cryptosystem: need to define functions for key generation, encryption, decryption, **and evaluation**

HOMOMORPHIC ENCRYPTION IN THE CLIENT-SERVER SETTING⁴



⁴A. Acar, H. Aksu, A. S. Uluagac, et al., "A Survey on Homomorphic Encryption Schemes: Theory and Implementation," *ACM Comput. Surv.*, vol. 51, no. 4, 79:1–79:35, Jul. 2018. DOI: 10.1145/3214303. [Online]. Available: <http://doi.acm.org/10.1145/3214303>.

2. HOMOMORPHISM

- In algebra: homomorphism = structure-preserving map between two groups
- What is a group?
 - A group is a set G plus an operation $\circ$
 - $\circ$ is used to combine two group elements a and b to form another element $a \circ b$
 - Closure: for all a, b in G : $a \circ b$ is also in G
 - Associativity: for all a, b, c in G : $(a \circ b) \circ c = a \circ (b \circ c)$
 - Identity element: G has a unique element e for which $e \circ a = a \circ e = a$
 - Inverse element: for each a in G , there exists b in G so that $a \circ b = b \circ a = e$
- Examples of groups?

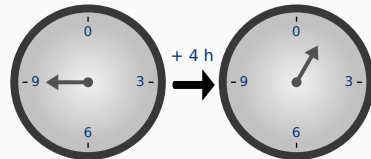
⁵X. Yi, R. Paulet, and E. Bertino, *Homomorphic Encryption and Applications* (SpringerBriefs in Computer Science). Cham: Springer International Publishing, 2014. DOI: 10.1007/978-3-319-12229-8. [Online]. Available: <https://link.springer.com/10.1007/978-3-319-12229-8>.

- In algebra: homomorphism = structure-preserving map between two groups
- What is a group?
 - A group is a set G plus an operation $\circ$
 - $\circ$ is used to combine two group elements a and b to form another element $a \circ b$
 - Closure: for all a, b in G : $a \circ b$ is also in G
 - Associativity: for all a, b, c in G : $(a \circ b) \circ c = a \circ (b \circ c)$
 - Identity element: G has a unique element e for which $e \circ a = a \circ e = a$
 - Inverse element: for each a in G , there exists b in G so that $a \circ b = b \circ a = e$
- Examples of groups?
 - Integers with addition
 - Nonzero rational numbers with multiplication
 - Modular arithmetic with modulus n and addition forms group $\mathbb{Z}_n$

⁵X. Yi, R. Paulet, and E. Bertino, *Homomorphic Encryption and Applications* (SpringerBriefs in Computer Science). Cham: Springer International Publishing, 2014. DOI: 10.1007/978-3-319-12229-8. [Online]. Available: <https://link.springer.com/10.1007/978-3-319-12229-8>.

MODULAR ARITHMETIC?

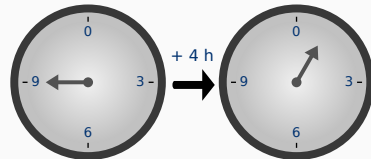
- Integer operations “wrap around” when reaching the modulus
- $9 + 4 \pmod{12} = 1$
- Result of each operation is the **remainder** of the result divided by the modulus
- Identity element? $9 + e \pmod{12} = 9$



Source:
https://commons.wikimedia.org/wiki/File:Clock_group.svg

MODULAR ARITHMETIC?

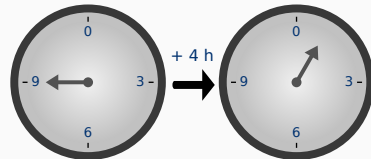
- Integer operations “wrap around” when reaching the modulus
- $9 + 4 \pmod{12} = 1$
- Result of each operation is the **remainder** of the result divided by the modulus
- Identity element? $9 + e \pmod{12} = 9$
 - $e = 0$
 - If our operation was multiplication instead of addition, what would be the identity element?
- Inverse element? $9 + i \pmod{12} = e$



Source:
https://commons.wikimedia.org/wiki/File:Clock_group.svg

MODULAR ARITHMETIC?

- Integer operations “wrap around” when reaching the modulus
- $9 + 4 \pmod{12} = 1$
- Result of each operation is the **remainder** of the result divided by the modulus
- Identity element? $9 + e \pmod{12} = 9$
 - $e = 0$
 - If our operation was multiplication instead of addition, what would be the identity element?
- Inverse element? $9 + i \pmod{12} = e$
 - $i = 3$

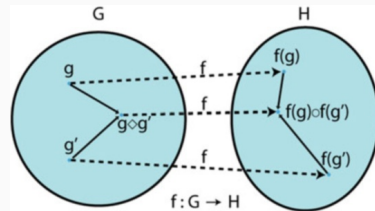


Source:
https://commons.wikimedia.org/wiki/File:Clock_group.svg

GROUP HOMOMORPHISM⁶

- What is a group homomorphism?
 - Given two groups $(G, \diamond)$ and $(H, \circ)$
 - A group homomorphism from $(G, \diamond)$ to $(H, \circ)$ is a function $f: G \rightarrow H$ such that for all g and g' in G :

$$f(g \diamond g') = f(g) \circ f(g')$$



⁶X. Yi, R. Paulet, and E. Bertino, *Homomorphic Encryption and Applications* (SpringerBriefs in Computer Science). Cham: Springer International Publishing, 2014. DOI: 10.1007/978-3-319-12229-8. [Online]. Available: <https://link.springer.com/10.1007/978-3-319-12229-8>.

- For a generic encryption scheme (P, C, K, E, D)
 - P, C : plaintext and ciphertext spaces
 - K : key space
 - E, D : encryption and decryption algorithms
- Let's assume the plaintexts and ciphertexts form groups $(P, \diamond)$ and $(C, \circ)$
- Then E maps from group P to C , i.e., $E_k : P \rightarrow C$, where $k \in K$ is either secret key (symmetric crypto) or public key (asymmetric crypto)
- The encryption is homomorphic, if for all a, b in P and k in K :

$$E_k(a) \circ E_k(b) = E_k(a \diamond b)$$

RSA IS A HOMOMORPHIC ENCRYPTION SCHEME⁸

- Note: this is **unpadded** RSA, which is **insecure**⁷
- We have the public key $pk = (n, e)$
- Plaintexts and ciphertexts from groups $(P, \cdot)$ and $(C, \cdot)$, where $\cdot$ is modular multiplication
- Encryption of plaintext m with public key pk : $E(m, pk) = m^e \pmod{n}$
- For plaintexts m_1, m_2 in P :

$$\begin{aligned} E(m_1, pk) \cdot E(m_2, pk) &= m_1^e \cdot m_2^e \pmod{n} \\ &= (m_1 \cdot m_2)^e \pmod{n} \\ &= E(m_1 \cdot m_2, pk) \end{aligned}$$

- This means: unpadded RSA has the homomorphic property, but for multiplication only (it is **partially homomorphic**)

⁷Padding with random bits makes it secure, but destroys the homomorphic property

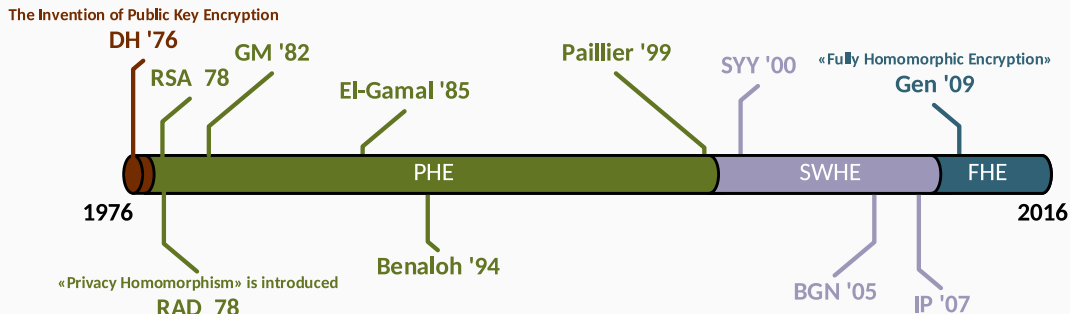
⁸R. L. Rivest, A. Shamir, and L. Adleman, "A method for obtaining digital signatures and public-key cryptosystems," *Communications of the ACM*, vol. 21, no. 2, pp. 120–126, Feb. 1978. DOI: 10.1145/359340.359342. [Online]. Available: <https://dl.acm.org/doi/10.1145/359340.359342>.

- How to determine if an encryption scheme is secure?
- Standard method: select a modern security definition, then prove that the adversary cannot distinguish between two ciphertexts
- Semantic security (or IND-CPA: indistinguishability under chosen plaintext attack)
 - Adversary is given a ciphertext c which encrypts either m_1 or m_2
 - Adversary is allowed to choose the two plaintexts m_1 and m_2
 - Adversary runs in polynomial time
 - Adversary guesses correctly with probability $1/2 + \epsilon$
 - ϵ is called the adversary's advantage
 - If the scheme is IND-CPA, then ϵ must be negligible
- Can a *deterministic* encryption scheme be semantically secure?

- Remember Kerckhoff's principle: an encryption scheme must be secure even if the adversary knows the algorithm and all parameters
- Deterministic encryption: same ciphertext is generated each time a message is encrypted
- In IND-CPA: adversary knows the plaintexts, so could simply encrypt them to decide whether m_1 or m_2 corresponds to the ciphertext c
- Deterministic encryption can **never** be semantically secure!
- All modern secure encryption schemes are **probabilistic**

3. PARTIALLY HOMOMORPHIC ENCRYPTION (PHE)

TIMELINE OF HOMOMORPHIC ENCRYPTION SCHEMES PRIOR TO FHE⁹



⁹A. Acar, H. Aksu, A. S. Uluagac, et al., "A Survey on Homomorphic Encryption Schemes: Theory and Implementation," *ACM Comput. Surv.*, vol. 51, no. 4, 79:1–79:35, Jul. 2018. DOI: 10.1145/3214303. [Online]. Available: <http://doi.acm.org/10.1145/3214303>.

ELGAMAL ENCRYPTION SCHEME

- Key generation:
 - Generate an efficient description of a cyclic group G , of order q , with generator g
 - Choose an integer x randomly from $\{1, \dots, q - 1\}$.
 - Compute $h = g^x$
 - Public key: (G, q, g, h) , private key: x
- Encryption:
 - Map message M to an element m in G
 - Choose an integer y randomly from $\{1, \dots, q - 1\}$
 - Compute $c_1 = g^y$ and $c_2 = m \cdot h^y$. Ciphertext: $E(M) = (c_1, c_2)$
- Decryption:
 - $s = c_1^x$
 - $m = c_2 \cdot s^{-1}$
- Evaluation:

$$\begin{aligned} E(m_1) \cdot E(m_2) &= (g^{y_1}, m_1 \cdot h^{y_1})(g^{y_2}, m_2 \cdot h^{y_2}) \\ &= (g^{y_1+y_2}, (m_1 \cdot m_2)h^{y_1+y_2}) = E(m_1 \cdot m_2) \end{aligned}$$

- Unpadded RSA (1977): multiplication
- Goldwasser-Micali (1982): addition modulo 2 (exclusive OR)
- ElGamal (1985): multiplication
- Benaloh (1994): addition modulo c (limited blocksize)
- Paillier (1999): addition and multiplication with a plaintext constant
- Boneh–Goh–Nissim (2005): addition and **one** multiplication

- Useful whenever the algorithm includes only addition or multiplication operations
- Genomic Testing for Disease Susceptibility¹⁰
- E-voting
- Private information retrieval

¹⁰G. Danezis and E. De Cristofaro, "Fast and Private Genomic Testing for Disease Susceptibility," in *13th Workshop on Privacy in the Electronic Society (WPES '14)*, Scottsdale, AZ, USA: ACM, Nov. 2014, pp. 31–34. DOI: 10.1145/2665943.2665952. [Online]. Available: <http://www.emilianodc.com/PAPERS/WPES14.pdf>.

4. SOMEWHAT HOMOMORPHIC ENCRYPTION (SHE)

- In algebra: a *ring* is similar to a group, but with two operations (e.g., addition and multiplication) (+ some additional axioms)
- Given two rings $(G, +, \cdot)$ and $(H, \boxplus, \boxtimes)$, $h : G \rightarrow H$ is a ring homomorphism if for all x and y in G :

$$h(x + y) = h(x) \boxplus h(y) \text{ and } h(x \cdot y) = h(x) \boxtimes h(y)$$

- From a computational point of view, why are rings superior to groups? (Why is it useful to have sums **and** products?)

A SIMPLE SOMEWHAT HOMOMORPHIC ENCRYPTION SCHEME¹¹

- Key generation: choose large **odd** number p
- Encryption of a bit b :
 - Choose random large multiple of p , e.g., qp
 - Choose random small number $2r + b$ (so that $r < p/2$)
 - $2r + b$ is called “noise”
 - Observe that $2r + b$ is even if $b = 0$, and odd if $b = 1$
 - Ciphertext $c = qp + 2r + b$
- Decryption:
 - $c \pmod{p} = 2r + b$
 - We can recover the noise by taking the ciphertext modulo p because $qp \pmod{p} = 0$
 - $b = (c \pmod{p}) \pmod{2}$
 - Taking the noise modulo 2 gives us the parity of the noise, which is exactly our bit b

¹¹M. van Dijk, C. Gentry, S. Halevi, *et al.*, “Fully Homomorphic Encryption over the Integers,” in *Advances in Cryptology – EUROCRYPT 2010*, H. Gilbert, Ed., ser. Lecture Notes in Computer Science, Berlin, Heidelberg: Springer, 2010, pp. 24–43. DOI: 10.1007/978-3-642-13190-5_2.

EXAMPLE APPLICATION OF THE SIMPLE SOMEWHAT HOMOMORPHIC ENCRYPTION

- Key generation: $p = 17$
- Encryption of bit 1:
 - Choose $q = 2$
 - Choose $r = 3$
 - Ciphertext $c = qp + 2r + b = 2 \times 17 + 2 \times 3 + 1 = 34 + 6 + 1 = 41$
- Decryption:
 - $b = (c \pmod{p}) \pmod{2} = (41 \pmod{17}) \pmod{2} = 7 \pmod{2} = 1$

EXAMPLE APPLICATION OF THE SIMPLE SOMEWHAT HOMOMORPHIC ENCRYPTION

- Key generation: $p = 17$
- Encryption of bit 1:
 - Choose $q = 2$
 - Choose $r = 3$
 - Ciphertext $c = qp + 2r + b = 2 \times 17 + 2 \times 3 + 1 = 34 + 6 + 1 = 41$
- Decryption:
 - $b = (c \pmod{p}) \pmod{2} = (41 \pmod{17}) \pmod{2} = 7 \pmod{2} = 1$
- What would have happened if we chose $r = 12$?

EXAMPLE APPLICATION OF THE SIMPLE SOMEWHAT HOMOMORPHIC ENCRYPTION

- Key generation: $p = 17$
- Encryption of bit 1:
 - Choose $q = 2$
 - Choose $r = 3$
 - Ciphertext $c = qp + 2r + b = 2 \times 17 + 2 \times 3 + 1 = 34 + 6 + 1 = 41$
- Decryption:
 - $b = (c \pmod{p}) \pmod{2} = (41 \pmod{17}) \pmod{2} = 7 \pmod{2} = 1$
- What would have happened if we chose $r = 12$?
 - This is against the rules!
 - Because we would get $c = 59$ and the decryption result is $(59 \pmod{17}) \pmod{2} = 8 \pmod{2} = 0$
 - So the noise $2r + b$ truly needs to be “small”!

WHY IS THIS SCHEME SECURE?

- Imagine we had no noise ($r = 0$)
 - If the adversary gets two encryptions of 0 (q_1p and q_2p)
 - They can recover the secret key $p = \text{GCD}(q_1p, q_2p)$
- But with noise
 - The GCD attack doesn't work
 - And actually, an efficient attack does not exist (as far as we know)
 - This is called the *approximate GCD assumption*

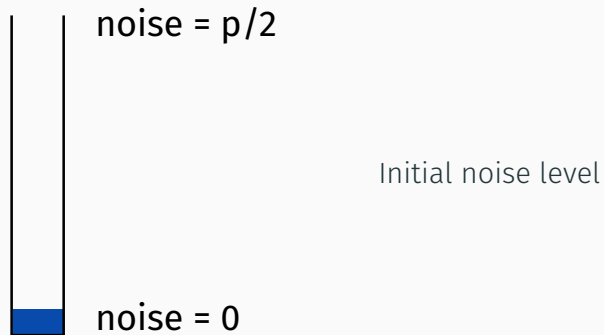
HOW HOMOMORPHIC IS THE SCHEME?

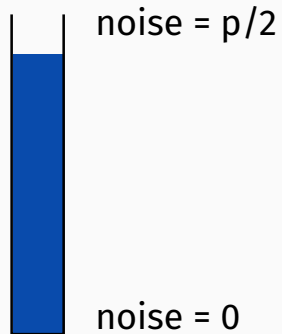
- Addition:
 - $c_1 + c_2 = (q_1 + q_2)p + 2(r_1 + r_2) + (b_1 + b_2)$
 - We can recover the sum of the two bits, $b_1 + b_2$, only if $r_1 + r_2 < p/2$
 - Observe that the noise after addition is **double** the initial noise
- Multiplication:
 - $c_1 c_2 = p(c_2 q_1 + c_1 q_2 - q_1 q_2) + 2(2r_1 r_2 + r_1 b_2 + r_2 b_1) + b_1 b_2$
 - We can recover the product of the two bits, $b_1 b_2$, only if $2(2r_1 r_2 + r_1 b_2 + r_2 b_1) < p/2$
 - Observe that the noise after multiplication is the **square** of the initial noise
- If we chain many operations to realize a useful function, **the noise will grow**, and as soon as $|\text{noise}| > p/2$, the decryption will be incorrect

- This is why this scheme is “somewhat” homomorphic: we can do lots of additions and some multiplications (as long as the noise is small enough)
- For example, if we choose $r \approx 2^n$, $p \approx 2^{n^2}$, $q \approx 2^{n^5}$, then we can evaluate polynomials of degree $\approx n$
- Already enough for many useful tasks:
 - Database search
 - Spam filtering

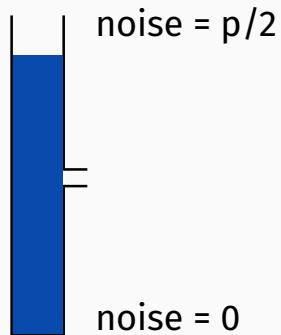
5. FULLY HOMOMORPHIC ENCRYPTION (FHE)

THE BOOTSTRAPPING METHOD

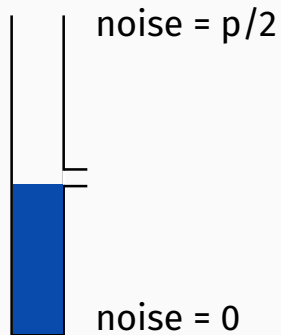




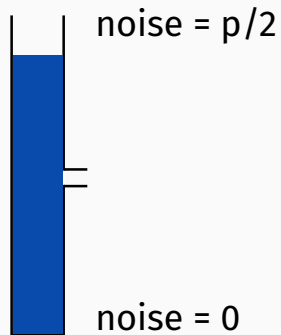
Noise level after some additions and multiplications



Desired mechanism: a “valve” that can reduce the noise to a fixed level

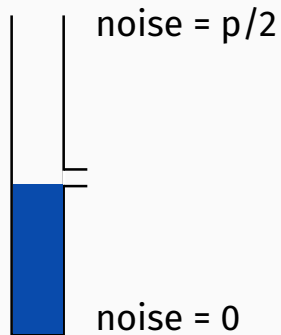


Desired mechanism: a “valve” that can reduce the noise to a fixed level



Desired mechanism: a “valve” that can reduce the noise to a fixed level
... so that we can perform more additions and multiplications

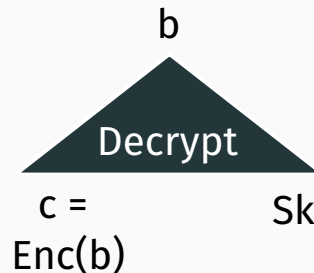
THE BOOTSTRAPPING METHOD



Desired mechanism: a “valve” that can reduce the noise to a fixed level
... so that we can perform more additions and multiplications
... before we rinse and repeat!

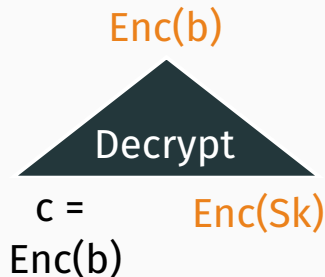
WHAT CAN WE USE AS NOISE-REDUCTION PROCEDURE?

- Key observation: if we decrypt the ciphertext, we recover the message and remove all noise
- Of course we cannot release the secret key, so the server cannot do this



WHAT CAN WE USE AS NOISE-REDUCTION PROCEDURE?

- Key observation: if we decrypt the ciphertext, we recover the message and remove all noise
- Of course we cannot release the secret key, so the server cannot do this
- But if we release an encryption of the secret key, the server can homomorphically evaluate the decryption function and get a fresh encryption of the ciphertext!
- The fresh encryption has a fixed noise level



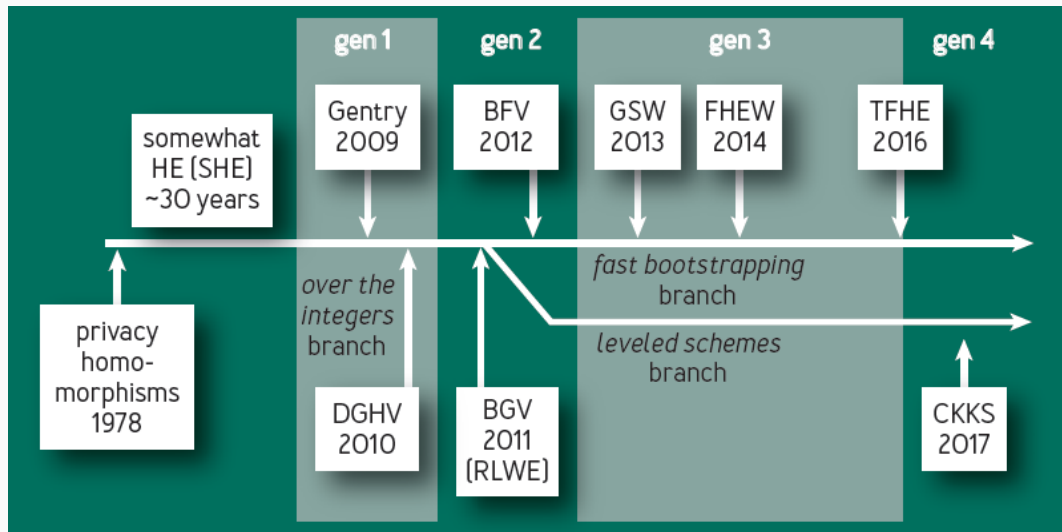
THE RECRYPT PROCEDURE, FORMALLY

- Let's assume we have a somewhat homomorphic public-key encryption scheme with keys pk and sk
- We have ciphertext c_1 , encrypted under pk_1 : $c_1 = Enc(m, pk_1)$
- We also have the secret key sk_1 , encrypted under pk_2 : $\overline{sk_1} = Enc(sk_1, pk_2)$
- Then we can define a **recrypt** procedure:
 - Encrypt ciphertext c_1 under pk_2 : $\overline{c_1} = Enc(c_1, pk_2)$
 - Homomorphically evaluate the decryption function with $\overline{c_1}$ and $\overline{sk_1}$, which results in an encryption of c_1 under pk_2 : $c = Enc(Dec(\overline{sk_1}, \overline{c_1}), pk_2) = Enc(c_1, pk_2)$
 - Then: c is a fresh encryption where the noise level of c_1 has been reset

- Gentry's bootstrapping theorem¹²: if a somewhat homomorphic encryption scheme can evaluate its own decryption function, then it is *bootstrappable* and can evaluate everything – with no limits on the number of additions or multiplications

¹²C. Gentry, "Computing arbitrary functions of encrypted data," *Communications of the ACM*, vol. 53, no. 3, pp. 97–105, Mar. 2010. doi: 10.1145/1666420.1666444. [Online]. Available: <https://dl.acm.org/doi/10.1145/1666420.1666444>.

CURRENTLY KNOWN FHE SCHEMES



- With FHE, we can let servers evaluate **any computable function** without giving the server access to the data!
- Why isn't everyone using it?

Many open source implementations with different features are available:

Table 1: Overview of HE libraries in terms of the supported schemes, domains (i.e., Integer, Binary, and Floating-point), levels/bootstrapping, and batching operations.

Library	Language	Domains	Schemes				
			BGV/BFV		CKKS		GSW
			Leveled/Fully [†]	Batching [‡]	Leveled/Fully [†]	Batching [‡]	Leveled/Fully [†]
Concrete [26]	Rust	Bin, FP	○	○	○	○	●
FHEW [35]	C, C++	Bin	○	○	○	○	●
HEAAN [23]	C, C++	FP	○	○	●	●	○
HElib [48]	C, C++	Int, Bin, FP	●	●	○	●	○
Lattigo [78]	Go	Int, FP	○	○	●	●	○
PALISADE [69]	C, C++	Int, Bin, FP	○	○	○	●	●
SEAL [72]	C, C++, .NET	Int, FP	○	○	○	●	○
TFHE [25]	C, C++	Bin	○	○	○	○	●

[†] For Leveled (LHE) and Fully (FHE) we categorize the libraries in four classes: ○ No support for either, ○ Support for LHE only, ● Support for FHE only, and ● Support for both LHE and FHE.

[‡] For batching, we classify the libraries as: ○ No support for batching, ○ Restricted modulus (i.e., in order to enable batching, the plaintext modulus p must satisfy $(p - 1) \mid m$, where m is the degree of the cyclotomic polynomial), and ● Flexible batching (i.e., no restrictions on p).

¹³C. Gouert, D. Mouris, and N. Tsoutsos, “SoK: New Insights into Fully Homomorphic Encryption Libraries via Standardized Benchmarks,” *Proceedings on Privacy Enhancing Technologies*, vol. 2023, no. 3, pp. 154–172, 2023. [Online]. Available:

<https://petsymposium.org/popets/2023/popets-2023-0075.php>.

PERFORMANCE OF FHE: EARLY SCHEMES¹⁴

Scheme Information		Platform	Parameters		Running Times				
Implemented Scheme	Base Scheme	Software	Security Parameter, λ	Dimension, n	PK Size	KeyGen	Enc	Dec	Recrypt
GH11 (Gentry and Halevi 2011)	Gen09 (Gentry 2009)	C/C++	72	33,768	2.25GB	2.2 h	3 min (SWHE)	0.66 s (SWHE)	31 min
CMNT11 (Coron et al. 2011)	DGHV10 (Van Dijk et al. 2010)	Sage 4.5.3	72	7,897	802MB	43 min	2 min 57 s	0.05 s	14 min 33 s
CNT12 (with compressed PK) (Coron et al. 2012)	DGHV10 (Van Dijk et al. 2010)	Sage 4.7.2	72	7,897	10.3MB	10 min	7 min 15 s	0.05 s	11 min 34 s
CNT12 (leveled) (Coron et al. 2012)	DGHV10 (Van Dijk et al. 2010)	Sage 4.7.2	72	5,700	18MB	6 min 18 s	3.4 s	0.00 s	2 h 27 min

These numbers have improved for more recent FHE schemes.

¹⁴A. Acar, H. Aksu, A. S. Uluagac, et al., "A Survey on Homomorphic Encryption Schemes: Theory and Implementation," *ACM Comput. Surv.*, vol. 51, no. 4, 79:1–79:35, Jul. 2018. DOI: 10.1145/3214303. [Online]. Available: <http://doi.acm.org/10.1145/3214303>.

PERFORMANCE OF FHE: CURRENT LIBRARIES¹⁵

Encoding	Binary Encoding (EncBIN) with word size 8 [*]						Integer Encoding (EncINT) [*]						Floating-Point Encoding (EncFP) [*]			
Operation Noise	Add. 	Mult. 	Eq. 	LT. 	ROT. 	Ctxt Size [†]	Add. 	Mult. 	Eq. 	LT. [‡] 	ROT. 	Ctxt Size	Add. 	Mult. 	ROT. 	Ctxt Size
HElib	 §					49.4						6.2				9.5
Lattigo						50.3						6.3				9.4
PALIS.						14.7						1.8				5.8
SEAL						14.6						1.8				4.5
TFHE						0.02	-	-	-	-	-	-	-	-	-	-
Fastest	TFHE	TFHE	TFHE	TFHE	TFHE [#]	N/A	PALIS.	PALIS.	PALIS.	PALIS.	Lattigo PALIS.	N/A	PALIS.	PALIS.	Lattigo PALIS.	N/A
Slowest	PALIS.	PALIS., SEAL	HElib, PALIS.	PALIS., SEAL	HElib	N/A	SEAL	HElib	HElib, SEAL	HElib, SEAL	HElib	N/A	Lattigo	HElib, Lattigo	HElib	N/A

[‡] Noise accumulation (or depth) relative to other HE operations, where the patterned bar indicates noise growth. For instance, binary addition results in significantly higher noise than integer addition (lower is better).

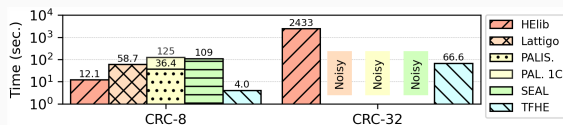
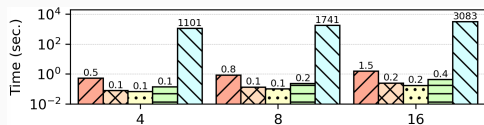
[§] The number of bars indicates latency relative to other HE operations on a logarithmic scale starting from 100 ms. For instance, any operation < 100 ms is awarded one bar 2 bars are < 1 second, and so on.

[†] Ciphertext size in MBs using the smallest parameters for each library to encrypt an 8-bit number and achieve depth 10 at 128 bits of security.

¹⁵C. Gouert, D. Mouris, and N. Tsoutsos, "SoK: New Insights into Fully Homomorphic Encryption Libraries via Standardized Benchmarks," *Proceedings on Privacy Enhancing Technologies*, vol. 2023, no. 3, pp. 154–172, 2023. [Online]. Available:

<https://petsymposium.org/popets/2023/popets-2023-0075.php>.

PERFORMANCE OF FHE: BENCHMARKS¹⁶



Left: linear regression with increasing number of attributes

Right: Cyclic redundancy check (CRC) over 8- and 32-bit numbers

¹⁶C. Gouert, D. Mouris, and N. Tsoutsos, "SoK: New Insights into Fully Homomorphic Encryption Libraries via Standardized Benchmarks," *Proceedings on Privacy Enhancing Technologies*, vol. 2023, no. 3, pp. 154–172, 2023. [Online]. Available:

<https://petsymposium.org/popets/2023/popets-2023-0075.php>.

IS FHE PERFECT NOW?

- What if we want our **function** to be private?
- What if we want to **verify** that the server has computed correctly?
- These are still active research questions!
- Also active research:
- Fixing vulnerabilities in FHE implementations
 - “Our proposed attack [...] can extract the entire message with a single power measurement¹⁷”
- Finding solutions that reduce ciphertext sizes
 - Hybrid homomorphic encryption¹⁸: encrypt data with symmetric encryption, encrypt symmetric key with HE, server converts symmetric ciphertext into HE ciphertext → reduces transmission volume for bandwidth-constrained clients

¹⁷F. Aydin, E. Karabulut, S. Potluri, et al., “RevEAL: Single-Trace Side-Channel Leakage of the SEAL Homomorphic Encryption Library,” in *2022 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, Mar. 2022, pp. 1527–1532. DOI: 10.23919/DATE54114.2022.9774724.

¹⁸K. Nguyen, M. Budzys, E. Frimpong, et al., “A Pervasive, Efficient and Private Future: Realizing Privacy-Preserving Machine Learning Through Hybrid Homomorphic Encryption,” in *2024 IEEE Conference on Dependable, Autonomic and Secure Computing (DASC)*, Nov. 2024, pp. 47–56. DOI: 10.1109/DASC64200.2024.00013. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/10798718>.

1. Cryptography refresher
2. Homomorphism
3. Partially homomorphic encryption (PHE)
4. Somewhat homomorphic encryption (SHE)
5. Fully homomorphic encryption (FHE)